Lab 3: Experiment Tracking and Model Management

MLflow Tracking and Model Registry

Submitted by	Mohd Faizy
Date	31 October 2026
Tools	MLflow 3.17.2, scikit-learn 1.9.1, Python 3.13, VS Code (PowerShell)
Dataset / model	Wine dataset (sklearn load_wine), GradientBoostingClassifier
Project folder	mlops-lab

1. Objective

Task 1 instruments a model training script with MLflow, runs it with several different hyperparameter sets, logs the accuracy of each run, and saves the trained model as an artifact. **Task 2** uses the MLflow Python API to register the highest-performing model, create a new version of the registered model, and move that version from the lifecycle stage "None" to "Staging".

2. Setup

MLflow and scikit-learn were installed into the project's virtual environment. The tracking URI is set to a local SQLite database (`sqlite:///mlflow.db`) because the Model Registry needs a database-backed store. Run metadata goes into `mlflow.db` and model files into the `mlruns` folder, both created in the project folder.

```
.venv\Scripts\activate
pip install mlflow scikit-learn
```

3. Task 1: MLflow Tracking

3.1 Training script (mlflow_train.py)

The script loads the Wine dataset, splits it 70/30 with a fixed seed, and trains a gradient boosting classifier once per hyperparameter set inside its own MLflow run. For every run it logs the hyperparameters (`log_params`), the test accuracy (`log_metric`), and the fitted model as an artifact (`log_model`).

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_wine
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)

mlflow.set_tracking_uri("sqlite:///mlflow.db")   # DB backend (needed for the registry)
mlflow.set_experiment("wine-lab3")
```

```
configs = [{"max_depth": 1, "learning_rate": 0.01},
           {"max_depth": 3, "learning_rate": 0.1},
           {"max_depth": 5, "learning_rate": 0.5}]

for p in configs:
    with mlflow.start_run():
        m = GradientBoostingClassifier(**p, n_estimators=10, random_state=42).fit(Xtr, ytr)
        acc = accuracy_score(yte, m.predict(Xte))
        mlflow.log_params(p)
        mlflow.log_metric("accuracy", acc)
        mlflow.sklearn.log_model(m, artifact_path="model", serialization_format="cloudpickle")
        print(p, "-> accuracy", round(acc, 4))
```

Three different hyperparameter sets were logged (the task requires at least two):

Run	max_depth	learning_rate	n_estimators	Logged
1	1	0.01	10	params, accuracy, model artifact
2	3	0.1	10	params, accuracy, model artifact
3	5	0.5	10	params, accuracy, model artifact

3.2 Issue encountered and fix

With MLflow 3.17.2 the first run failed with `MlflowException: The saved sklearn model references untrusted types ... sklearn.tree._tree.Tree`. MLflow 3.x saves scikit-learn models in the skops format by default, and skops does not trust the tree structure used inside gradient boosting models. The fix was to save the model with the pickle-based format instead, by adding `serialization_format="cloudpickle"` to `log_model` (line 25). The failed attempt's database and runs were deleted, and the script was run again.

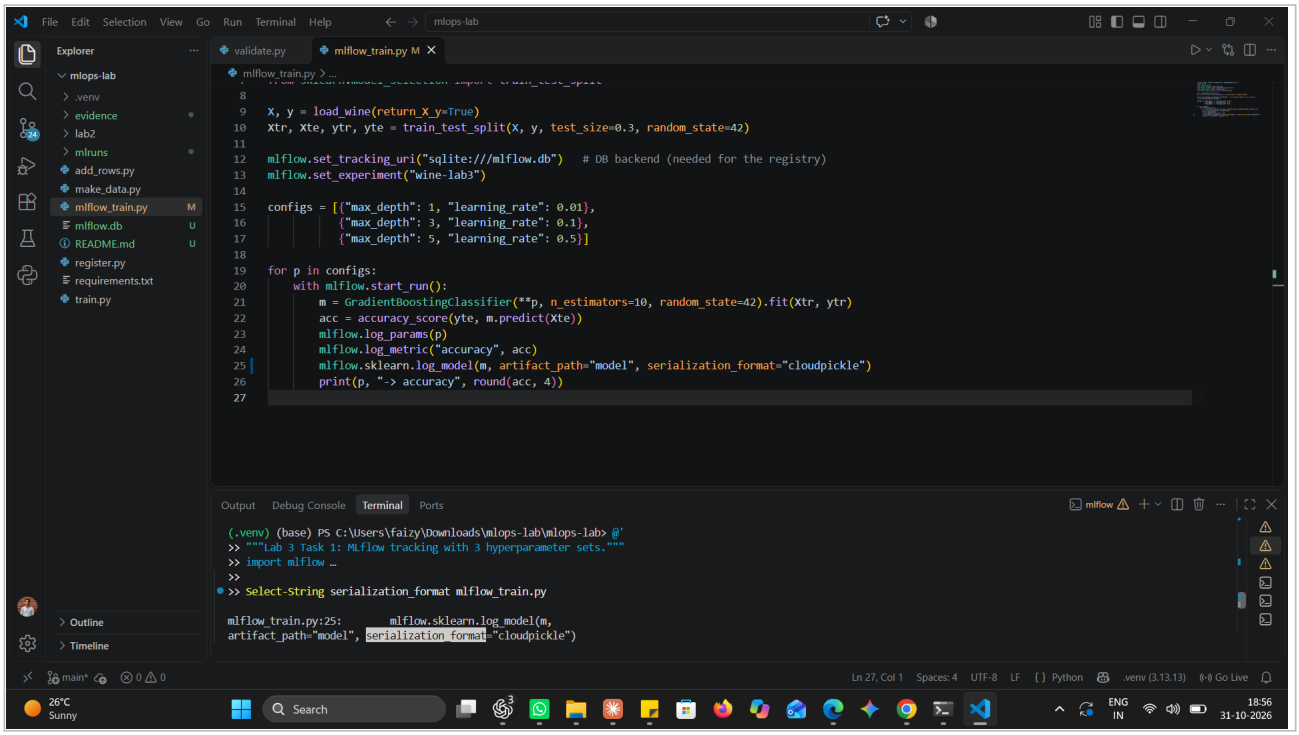


Figure 1: Updated mlflow_train.py in VS Code; the terminal confirms line 25 now contains serialization_format="cloudpickle" (taskbar shows the date and time).

3.3 Run output

After the fix the training script ran successfully and created `mlflow.db` and the `mlruns` folder (both visible in the Explorer panel of Figure 2). The terminal in Figure 2 shows the accuracy line of the last run, `{'max_depth': 5, 'learning_rate': 0.5} -> accuracy 0.8889`.

4. Task 2: Model Registry

4.1 Registration script (register.py)

The script (1) finds the run with the highest accuracy in the `wine-lab3` experiment, breaking ties by the most recent start time, (2) registers that run's model as `WineClassifier` (version 1), (3) registers the same model again to create a new version (version 2), and (4) transitions version 2 from the stage "None" to "Staging". It also sets the alias `staging` on version 2.

```
import mlflow
from mlflow.tracking import MlflowClient

mlflow.set_tracking_uri("sqlite:///mlflow.db")
client = MlflowClient()

exp = client.get_experiment_by_name("wine-lab3")
best = client.search_runs([exp.experiment_id],
                          order_by=["metrics.accuracy DESC", "attributes.start_time DESC"],
                          max_results=1)[0]
print("Best run:", best.info.run_id, best.data.params, best.data.metrics)

uri = f"runs:/{best.info.run_id}/model"
name = "WineClassifier"

v1 = mlflow.register_model(uri, name)     # version 1
v2 = mlflow.register_model(uri, name)     # version 2 (the "new version")
print("Registered versions:", v1.version, v2.version)

# Stage transition None -> Staging (deprecated in MLflow 2.9+ but still works)
client.transition_model_version_stage(name, v2.version, stage="Staging")
# Modern alternative: alias
client.set_registered_model_alias(name, "staging", v2.version)

mv = client.get_model_version(name, v2.version)
print(f"Version {mv.version} stage: {mv.current_stage}")
```

4.2 Output

The terminal in Figure 2 shows the registry step running:

- The best run was selected: run id `af64d12f0e754687bfa22548a7747a41` with `max_depth=5`, `learning_rate=0.5` and accuracy 0.8889.
- `Successfully registered model 'WineClassifier'` and `Created version '1' of model 'WineClassifier'`.
- `Registered model 'WineClassifier' already exists. Creating a new version of this model...`, which is the creation of the new version.

The warning that the run has no artifacts at path 'model' is informational: in MLflow 3 the model is stored as a logged model, so MLflow registers it from its `models:/` location instead.

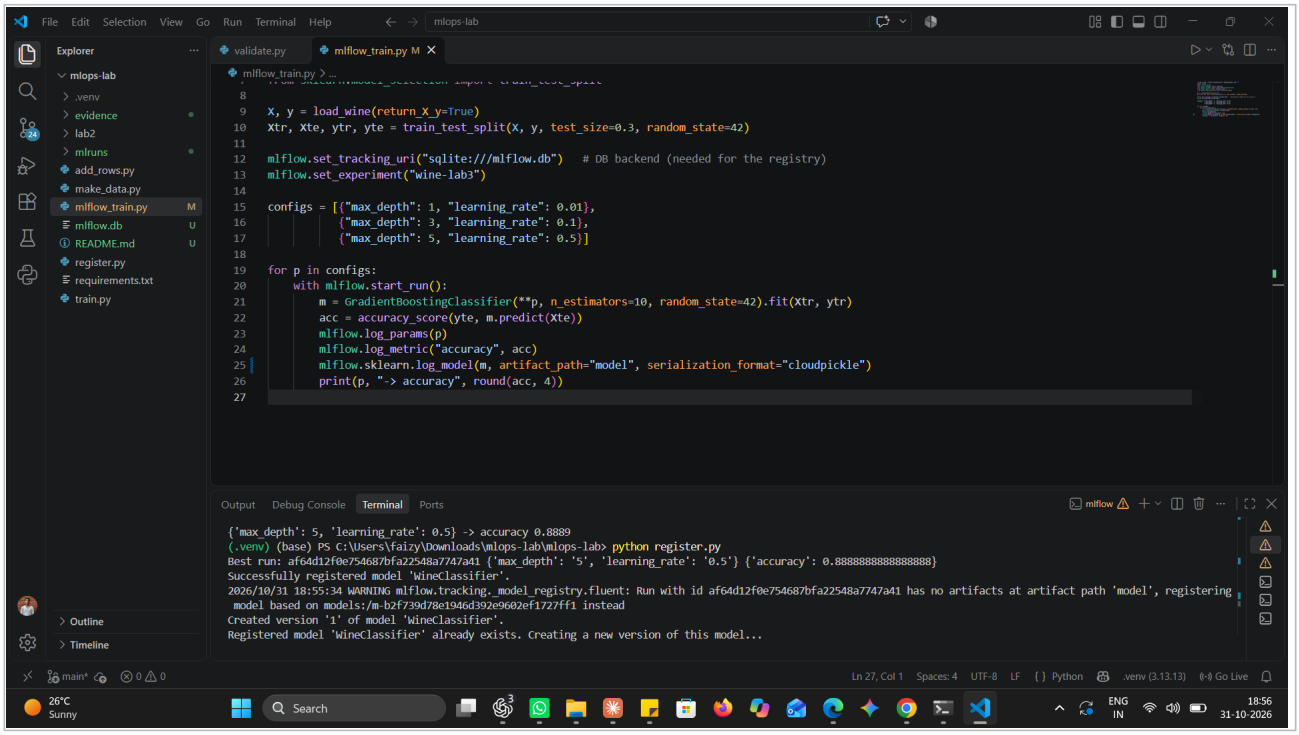


Figure 2: Terminal output of the last training run and of register.py (best run selected, WineClassifier version 1 created, new version being created). Taskbar shows 31-10-2026.

4.3 Lifecycle stage

A newly created model version starts in the stage "None". The call `transition_model_version_stage(name, version, stage="Staging")` moves a version to "Staging", and the script prints the resulting stage with `get_model_version(...).current_stage`. MLflow has deprecated stages in favour of aliases since version 2.9, so the script also assigns the alias `staging`, which is shown on the version in the MLflow UI. The UI is started with:

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

and opened at `http://127.0.0.1:5000`. The experiment page lists the runs with their parameters and accuracy, each run's Artifacts tab holds the saved model, and the Models tab shows the registered model with its versions.

5. Conclusion

MLflow tracking recorded the hyperparameters, accuracy and trained model of each run, which made it possible to compare configurations and pick the best one programmatically. The Model Registry then turned that run into a named, versioned model (`WineClassifier`) with a new version prepared for promotion to Staging. One practical lesson was that MLflow 3.x changed the default scikit-learn serialization to skops, which needs an explicit `serialization_format="cloudpickle"` for tree-based models.